UNIVERSIDADE ESTADUAL DO CENTRO-OESTE
PROGRAMAÇÃO ORIENTADA A OBJETOS II
TIAGO HENRIQUE PEREZ

Programação Orientada a Objetos II

Trabalho 2

Equipe: Tiago Henrique Perez

1- Diagnóstico

O projeto original consiste em um sistema de mantimento e gerenciamento de anotações chamado *InfoKeep*, onde um usuário pode realizar ações como:

- Criar uma conta;
- Fazer login em uma conta já existente;
- Criar uma nota;
- Excluir uma nota;
- Fixar uma nota;
- Editar uma nota.

Estas ações são possíveis graças ao banco de dados integrado ao sistema por meio do padrão DAO (Data Access Object). O sistema também utiliza o padrão de arquitetura de software MVC (Model-View-Controller), que divide a aplicação em três camadas a fim de facilitar a manutenção de camadas individuais, sem a necessidade de alterar todas elas de uma só vez.

2- Novas Funcionalidades

O trabalho 2 da disciplina visa adicionar no projeto já existente o padrão arquitetural Service Layer, juntamente de três padrões de projeto GoF.

Dentre os oito padrões GoF analisados, foram escolhidos os seguintes padrões que mais se adequaram ao sistema:

- Builder
- Strategy
- Command

O padrão arquitetural já estava implementado no projeto original, mas é crucial que ele seja refatorado a um estado mais completo e que seja entendido sua utilidade no código.

O Service Layer resolve um problema que aparece quando o Controller faz coisa demais: ele acaba misturando interface, regra de negócio e acesso ao banco. Separando em camadas, cada classe tem uma responsabilidade só.

Os três padrões de projeto GoF foram escolhidos cada um com sua finalidade específica:

Command:

Problema no código: NotaController repetia o mesmo bloco try/catch e voltarParaMenu() em salvar, excluir e fixar.

Necessidade do app: o Keep tem “desfazer”. Sem isso, uma exclusão acidental é definitiva, e para o usuário é o pior erro que o app pode cometer.

Strategy:

Problema no código: a ordem das notas estava fixa no SQL (ORDER BY fixado DESC, notaid).

Necessidade do app: quem tem muitas notas quer ordenar por título ou ver as mais recentes primeiro.

Builder:

Problema no código: Nota tinha construtores posicionais (int, int, String, String, boolean), fáceis de trocar sem erro de compilação.

Necessidade criada pelo Command: o desfazer precisa copiar notas e editar só alguns campos, e o NotaBuilder.de(Nota original) faz isso.

3- Registro dos Padrões

● 3.1- Service Layer

Identificação:

Concentra as regras de negócio entre a interface e o banco de dados. Composto por pacote service (NotaService, UsuarioService, ServiceException), controller e DAO (INota, NotaDAO).

Situação Anterior:

A camada já existia, mas o NotaController decidia se a nota fixava ou desfixava (!nota.isFixado()), o NotaService não conferia o dono da nota ao editar, excluir ou fixar, e criava o DAO com new NotaDAO(), o que impedia testar sem banco.

Solução Implementada:

O NotaService passou a receber o DAO pelo construtor (NotaService(INota notaDAO)), a conferir o dono em toda alteração (buscarDoUsuario) e a decidir sozinho o estado de fixado: alternarFixado(notaID, userID) lê a nota, inverte fixado e grava. O controller só chama o serviço e mostra os alertas. O DAO ganhou buscarPorId e restaurar.

Justificativa e Verificação:

Regras em um só lugar, controllers sem lógica de negócio e serviço testável com qualquer INota. Evidência: a classe VerificacaoPadroes roda o serviço com DAO em memória e confirma que excluir, fixar e editar nota de outro usuário são recusados, que nota inexistente é recusada e que alternarFixado devolve o novo estado (6 verificações).

- ## 3.2- Builder

Identificação:

Monta objetos Nota nomeando cada campo. Classe nova NotaBuilder (pacote model); usada em NotaDAO, NotaController e nos comandos do Command.

Situação Anterior:

Nota tinha construtores posicionais (5 parâmetros: int, int, String, String, boolean; e 3 parâmetros). Trocar a ordem dos argumentos compilava sem erro e só aparecia como bug em execução. Também não havia como copiar uma nota.

Solução Implementada:

NotaBuilder com métodos encadeados notaID, userID, nome, texto, fixado e build(), mais NotaBuilder.de(original) para editar ou copiar uma nota existente. Exemplo no NotaController:
NotaBuilder.de(nota).nome(txtTitulo.getText()).texto(txtTexto.getText()).build(). No NotaDAO, o método mapear monta a nota a partir da linha do banco. A validação continua no NotaService.

```
// NotaDAO: linha do banco -> Nota
return new NotaBuilder()
        .notaID(rs.getInt("notaid"))
        .userID(rs.getInt("userid"))
        .nome(rs.getString("nome"))
        .texto(rs.getString("texto"))
        .fixado(rs.getBoolean("fixado"))
        .build();

// NotaController: editar só título e texto
Nota editada = NotaBuilder.de(nota)
        .nome(txtTitulo.getText()
        .texto(txtTexto.getText())
        .build();
```

Justificativa e Verificação:

Cada campo é lido na chamada, sem depender da ordem, e a cópia de nota ficou em um só lugar. Evidência: a VerificacaoPadroes confirma os valores padrão e que NotaBuilder de(...) gera cópia independente (2 verificações); no sistema, roda a cada listagem (DAO) e a cada salvamento.

- ## 3.3- Strategy

Identificação:

Permite escolher como as notas são ordenadas. Pacote novo strategy: EstrategiaOrdenacao, FixadasPrimeiro, PorTitulo, MaisRecentes; usado por NotaService, Sessao e MenuController.

Situação Anterior:

A ordem estava fixa na consulta SQL do NotaDAO (ORDER BY fixado DESC, notaid). Ordenar por título ou pelas mais recentes exigiria novas consultas ou if no DAO.

Solução Implementada:

EstrategiaOrdenacao é a interface (nome() e ordenar(List<Nota>)); FixadasPrimeiro (padrão, igual ao comportamento original), PorTitulo e MaisRecentes são as estratégias concretas. O Context é NotaService.listar(userID, estrategia), que busca as notas e delega a ordem. O MenuController monta o menu “Ordenar por” e a escolha fica na Sessao. O DAO agora só devolve as notas por notaid. MaisRecentes usa o notaid como aproximação, pois não há data de criação no banco.

```
public List<Nota> listar(int userID, EstrategiaOrdenacao estrategia) throws ServiceException {
    EstrategiaOrdenacao usada = estrategia != null ? estrategia : ordenacaoPadrao;
    try {
        return usada.ordenar(notaDAO.listarPorUsuario(userID));
    } catch (SQLException e) {
        throw traduzirFalha("listar", e)
    }
}
```

Justificativa e Verificação:

Novas ordenações entram criando uma classe, sem mexer no serviço, no DAO ou no SQL. Evidência: com as notas Charlie, alfa e Bravo (fixada), a VerificacaoPadroes obteve FixadasPrimeiro: Bravo* Charlie alfa; PorTitulo: alfa Bravo* Charlie; MaisRecentes: Bravo* alfa Charlie (5 verificações, todas aprovadas).

- ## 3.4- Command

Identificação:

Transforma cada ação sobre uma nota em um objeto que se executa e se desfaz, viabilizando o “Desfazer”. Pacote novo command: ComandoNota, SalvarNotaCommand, ExcluirNotaCommand, FixarNotaCommand, HistoricoComandos; usado por NotaController, MenuController e Sessao.

Situação Anterior:

Salvar, excluir e fixar eram três métodos do NotaController com o mesmo bloco try/catch e voltarParaMenu(). Nenhuma ação podia ser desfeita, então uma exclusão acidental era definitiva.

Solução Implementada:

ComandoNota é a interface (executar(), desfazer(), descricao()); os três comandos concretos guardam o necessário para desfazer (por exemplo, ExcluirNotaCommand guarda uma cópia da nota e desfazer() chama service.restaurar(copia)). O Receiver é o NotaService; o Invoker é o HistoricoComandos, uma pilha de até 20 comandos guardada na Sessao; o cliente é o NotaController, que cria o comando e chama Sessao.getHistorico().executar(comando). O menu Nota ganhou o item “Desfazer”.

```
// HistoricoComandos (Invoker)
public void executar(ComandoNota comando) throws ServiceException {
    comando.executar();
    executados.push(comando);
}

public void desfazer() throws ServiceException {
    ComandoNota ultimo = executados.peek();
    ultimo.desfazer();
    executados.pop();
}
```

Justificativa e Verificação:

Desfazer em qualquer ação, tratamento de erro sem repetição e novas ações desfazíveis com uma nova classe. Evidência: a VerificacaoPadroes executa uma cadeia de ações e as desfaz em ordem inversa: desfazer exclusão recria a nota com o mesmo ID, desfazer fixar desfixa, desfazer edição restaura título e texto, e um comando que falha na validação não entra no histórico (12 verificações; total geral de 25, todas aprovadas).